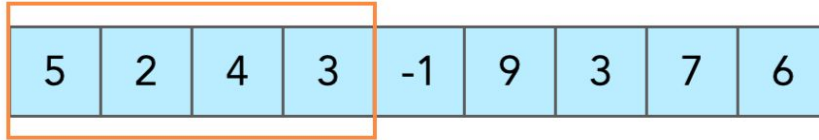


Sliding Window



Sliding window —> —>

Lecture Flow

- Pre-requisites
- Introduction and Basic Concepts
- One Dimensional problems
- Two Dimensional problems
- Variants
- Pitfalls
- Complexity Analysis
- Applications of sliding window
- Practice Questions



Pre-requisites

1. Arrays
2. Dictionary/Maps
3. Sets/Hash Sets
4. Two Pointers

Motivation Problem

Consider the problem of finding the maximum sum of 3 consecutive elements in an array.

How can we solve this problem ?



Can we just calculate the sum
for each sub array size 3 and
select the maximum sum?

Time complexity

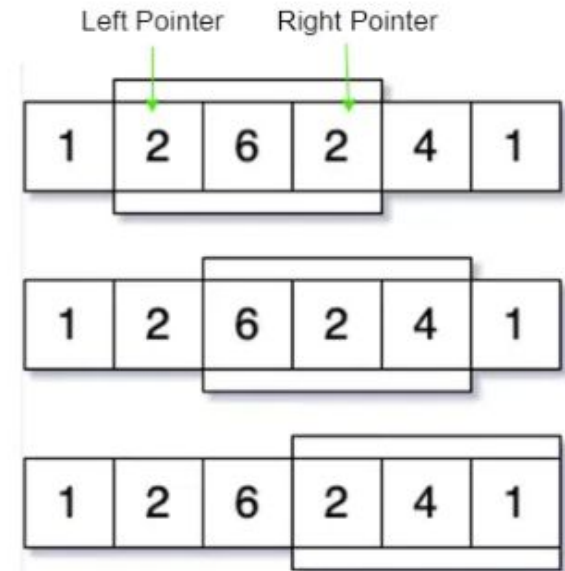
- We are calculating each subarray sum on every iteration
- We have n iteration and k element
- So the Time complexity is : **$O(n*k)$**



Can we do better ?

Introduction

- **Sliding Window** is a method used to efficiently solve problems that involve defining a window in the input data (**arrays** or **strings**)
- Then moving that window across the data to perform some operation within the window
- It Unlocks solutions for finding max/min values, counting elements, and pattern detection



Basic Concepts

The sliding window technique consists of two key components: **window size** and **window movement**.

- **window size** - the number of elements in the array or string that the window encompasses.
- **window movement** - the number of elements by which the window moves after each step.

One Dimensional problems

One-dimensional problems can be solved using the sliding window technique by creating a window of fixed size that moves over the array, one element at a time.

One Dimensional problems Cont.

Consider the problem of finding the maximum sum of k consecutive elements in an array.

How can we use sliding window to solve this problem ?

One Dimensional Sliding Window Simulation

2	1	4	3	0
---	---	---	---	---

$K = 2$

One Dimensional Sliding Window Simulation

Step: 1

2	1	4	3	0
---	---	---	---	---

maximum = $2+1 = 3$

$K = 2$

One Dimensional Sliding Window Simulation

Step: 2



maximum = $1+4 = 5$

$K = 2$

One Dimensional Sliding Window Simulation

Step: 3



maximum = $4+3 = 7$

$K = 2$

One Dimensional Sliding Window Simulation

Step: 4

2	1	4	3	0
---	---	---	---	---

maximum = $3+0 = 3$

Maximum sum of two items from this array is 7 and the pair is {4,3}.

$K = 2$

Variants

There are several types of sliding window:

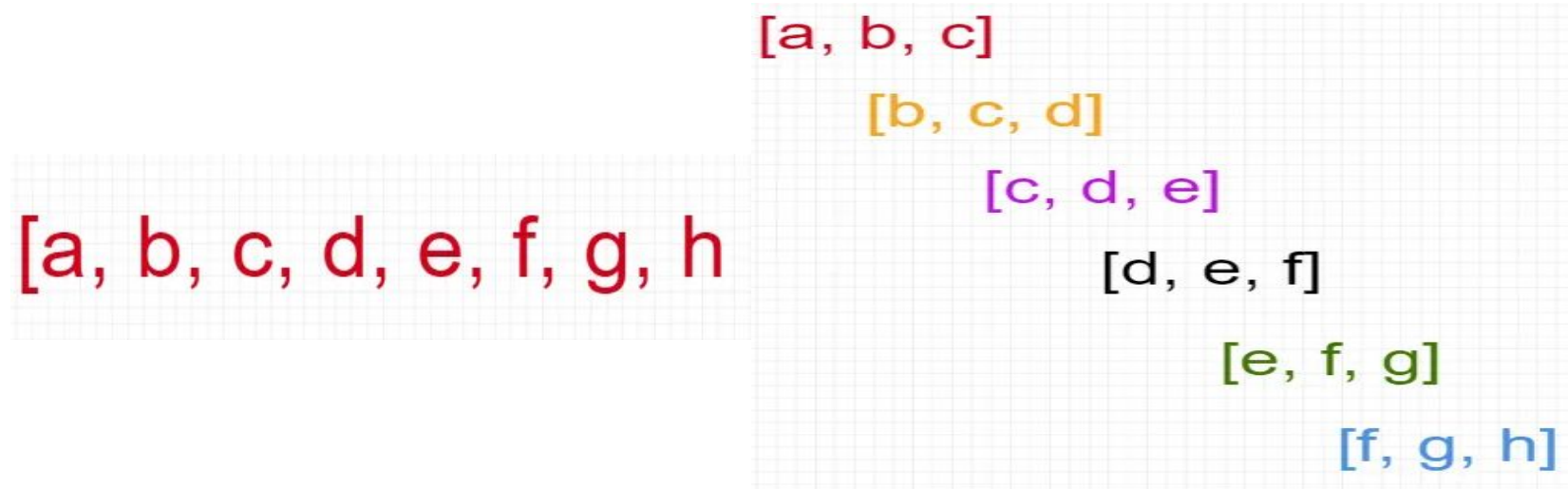
1. Fixed Window Length k
2. Dynamic Sliding Window

Variant I

Fixed Size Sliding Window: Optimization

In this technique, a window of fixed size is moved over the input data, and some operation is performed on the data within the window.

For $k = 3$:



Fixed Size Sliding Window: Optimization



Example: Given an array of integers, find the maximum sum of any contiguous subarray of size 3.

Optimization

- What is the difference between those two subarrays ?
- Can we construct the second one from the first one?
- If so how can we do that ?

[3, 1, 4, 2, 3, 0]

[3, 1, 4, 2, 3, 0]

Fixed Size Sliding Window: Optimization

`max_sum = 0`

`[3, 1, 4, 2, 3, 0]`

Fixed Size Sliding Window: Optimization



`max_sum = 8`

`[3, 1, 4, 2, 3, 0]`

Fixed Size Sliding Window: Optimization

max_sum = 8

[3, 1, 4, 2, 3, 0]

Fixed Size Sliding Window: Optimization



`max_sum = 9`

`[3, 1, 4, 2, 3, 0]`

Fixed Size Sliding Window: Optimization



`max_sum = 9`

`[3, 1, 4, 2, 3, 0]`



Practice Question

[Problem Link](#)



Time and space complexity?



Fixed-size sliding window

- Time complexity: $O(n)$
- Space complexity: $O(1)$



Variant II

Dynamic Sliding Window

In this technique, the size of the window is **not** fixed, and it can be increased or decreased based on certain conditions.

For example, given a string, find the longest substring with unique characters.

Dynamic Sliding Window

s = "abcbad"

`longest = 0`

Dynamic Sliding Window

s = "abcbad"

longest = 1

Dynamic Sliding Window

s = "abcbad"

longest = 2

Dynamic Sliding Window

s = "abcbad"

longest = 3

Dynamic Sliding Window

s = "abcbad"

```
longest = 3 // duplicate found, shrink window
```

Dynamic Sliding Window

s = "abcbada"

```
longest = 3 // duplicate found, shrink window
```

Dynamic Sliding Window

`s = "abcbad"`

`longest = 3`

Dynamic Sliding Window

`s = "abcbad"`

`longest = 3`

Dynamic Sliding Window

s = "abcbad"

longest = 4



Longest substring without repeating character

[Problem Link](#)



Time and space complexity?



Dynamic sliding window

- Time complexity: $O(n)$
- Space complexity: $O(1)$



Dynamic sliding window

Let's look at another question on dynamic sliding window.

[Problem Link](#)

General Template I

```
def dynamicwindowatmost(self, nums: str) -> int:
    n=len(nums)
    ans=0
    Window = # initialize
    left=0
    for right in range(n):
        Window = # update the window with right
        while window is invalid:
            Window = # remove the left part
            left += 1 # update left pointer

        ans= # update the answer

    return ans
```

General Template II

```
def dynamicwindowatleast(self, nums: str) -> int:
    n=len(nums)
    ans=0
    window= # initialize
    left=0
    for right in range(n):
        Window = # update the window with right
        while window is valid:
            ans= # update the answer
            Window = # remove the left part
            left += 1 # update left pointer

    return ans
```

Applications of sliding window

Sliding Window Problems

Sliding Window for Optimization

- Used when optimizing some metric (min/max value, longest/shortest condition, etc.).
- Example: Longest substring with at most k distinct characters.

Sliding Window for Counting

- Used when the problem involves counting occurrences of elements within a window.
- Example: Counting anagrams of a string in another string.

Sliding Window: Counting

In this technique, a sliding window is used to count the number of occurrences of a particular item in the input data.

For example, given two strings s and p , find the number of anagrams of p in s .

Sliding Window: Counting

s = "cbadabac"

p = "abc"

Sliding Window: Counting

`anagrams = 0`

`s = "cbadabac"`

`target = { a:1, b:1, c:1 } // represent p as a counter`

Sliding Window: Counting

anagrams = 1

s = "cbadabac"

```
target = { a:1, b:1, c:1 }
```

```
window = { a:1, b:1, c:1 } // represent each window as a  
counter
```

52 and compare to target

Sliding Window: Counting

anagrams = 1

s = "cbadabac"

target = { a:1, b:1, c:1 }

window = { a:1, b:1, d:1 }

Sliding Window: Counting

anagrams = 1

s = "cbadabac"

target = { a:1, b:1, c:1 }

window = { a:2, d:1 }

Sliding Window: Counting

anagrams = 1

s = "cbadabac"

target = { a:1, b:1, c:1 }

window = { a:1, b:1, d:1 }

Sliding Window: Counting

anagrams = 1

s = "cbadabac"

target = { a:1, b:1, c:1 }

window = { a:2, b:1 }

Sliding Window: Counting

anagrams = 2

s = "cbadabac"

target = { a:1, b:1, c:1 }

window = { a:1, b:1, c:1 }



Practice Question

[Problem Link](#)



Time and space complexity?



Fixed-size sliding window

- Time complexity: $O(n)$
- Space complexity: $O(1)$

Two Dimensional problems

Two-dimensional problems can be solved using the sliding window technique by moving a window over the matrix , one row or column at a time.

Applications of 2D Sliding Window:

1. Image Processing
2. Pattern Matching
3. Matrix-based DP Problems - Problems like "Maximum Sum Submatrix of Size $k \times k$ ".

Two Dimensional problems Cont.

Consider the problem of finding the maximum sum of a submatrix of size $k \times k$ in a matrix.

How can we solve this problem using sliding window ?

Two Dimensional problems Cont.

To solve this problem using the sliding window technique,

- We would create a window of size $k \times k$ and
- Move it over the matrix, summing up the elements in each window and
- Keeping track of the maximum sum we have seen so far.

Two Dimensional Sliding Window Simulation

5	3	1
2	4	2
0	3	1

Two Dimensional Sliding Window Simulation

Step: 1

5		3	1
2		4	2
0	3	1	

maximum = $5+3+2+4 = 14$

Two Dimensional Sliding Window Simulation

Step: 2

5	3 1	
2	4 2	
0	3	1

maximum = $3+1+4+2 = 10$

Two Dimensional Sliding Window Simulation

Step: 3

5	3	1
2	4	2
0	3	1

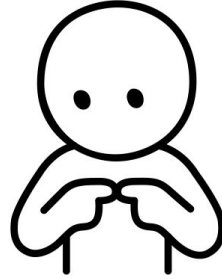
maximum = $2+4+0+3 = 9$

Two Dimensional Sliding Window Simulation

Step: 4

5	3	1
2	4	2
0	3	1

maximum = $4+2+3+1 = 10$



Common Pitfalls

Pitfalls

There are several common pitfalls when using sliding windows:

1. Off-by-one error
2. Not considering all possible cases
3. Not handling edge cases
4. Inefficient window updates
5. Not understanding the problem requirements

Complexity Analysis

- Fixed-size sliding window
 - Time complexity: **$O(n)$**
 - Space complexity: **$O(1)$**
- Dynamic sliding window
 - Time complexity: **$O(n)$**
 - Space complexity: **$O(1)$**
- Counting sliding window
 - Time complexity: **$O(n)$**
 - Space complexity: **$O(1)$**

Applications of sliding window technique

- Maximum Sum Subarray of Size K
- Distinct Elements in Every Window of Size K
- Maximum Consecutive Ones (with At Most K Flips)
- Anagram Search in a String
- Smallest Subarray with Sum $\geq K$
- Longest Subarray with Sum $\leq K$

Checkpoint

Practice Questions

- [Minimum Size Subarray Sum](#)
- [Longest Repeating Character Replacement](#)
- [Permutation in String](#)
- [Smallest Range Covering Elements from K Lists](#)
- [Shortest Subarray with Sum at Least K](#)
- [Fruit Into Baskets](#)
- [Longest Turbulent Subarray](#)
- [Get Equal Substrings Within Budget](#)
- [Minimum Window Substring](#)
- [Codeforce's practice problems](#)



Quote Of The Day

"Your big opportunity may be right where you are now. Open your eyes, and you'll find it in your windows."

- Napoleon Hill